École Polytechnique Fédérale de Lausanne

Adaptive Calibration Algorithms for Tunable Analog Peripherals

by Khalil M'hirsi

Master Thesis

Approved by the Examining Committee:

Prof. Mathieu Salzmann
Thesis Advisor

Richard Lengagne
External Expert

Christophe Constantin
Thesis Supervisor

EPFL CVLAB
BC 309 (Bâtiment BC)
Station 14
CH-1015 Lausanne

7 August 2026

“Pour ceux qui
viendront après.”
— Clair Obscur, Expedition 33

Acknowledgments

I would like to express my sincere gratitude to Christophe Constantin, my supervisor at Logitech, for his trust, guidance, and critical perspective throughout this thesis. I am equally thankful to Yael, a close collaborator on this project, whose feedback and thoughtful challenges helped sharpen both the ideas and the direction of the work. I am also deeply grateful to Professor Mathieu Salzmann for his valuable advice, high standards, and steady guidance in ensuring that this thesis remained rigorous and focused.

I would also like to thank Kilian and Frederic for their technical support on the hardware side, and Jozef and Lilou for the many helpful discussions on machine learning and signal processing. I am grateful as well to my teammates Anais and Kevin, to Thomas, and to the broader team at Logitech for the many forms of support that surrounded this work, from technical discussions and thoughtful advice to the practical organization of data-collection sessions and user tests. This thesis would not have taken its final shape without that collective effort.

On a more personal note, I am deeply grateful to my partner, Sophia, for the patience, steadiness, and care she brought throughout a demanding period. Her presence made the difficult moments lighter and the better ones meaningful. I am equally thankful to my close friends Hugo, Christelle, and Maxence, whose encouragement, listening, and honesty helped me keep moving forward when my own perspective narrowed. More broadly, I am grateful to those closest to me who stood by me through a difficult time and helped me carry this work to its conclusion.

Lausanne, 7 August 2026 Khalil M’hirsi

Abstract

Analog peripherals with continuous depth-sensing hardware offer sub-millimetre actuation precision, yet their firmware often relies on static thresholds that ignore per-user biomechanical variance and degrade under neuromotor fatigue. This thesis addresses that gap by designing an end-to-end click-sensitivity recommendation pipeline for Haptic Inductive Trigger System (HITS) devices, using telemetry streamed from the mouse in order to recommend personalized Actuation Point (AP) and Rapid Trigger (RT) settings. The system makes three contributions: (1) a supervised labeling workflow that first generates frame-level labels from algorithmic criteria (depth geometry, speed, and acceleration thresholds) and then iteratively improves those labels through model-assisted refinement; (2) a four-class temporal intent-detection model—supporting Random Forest, Gradient Boosted Trees, and a dilated residual fully-convolutional network—that combines digital signal preprocessing, class-imbalance-aware sample selection, and context-rich features to identify make-onset, click-hold, and break-onset phases from high-frequency telemetry; and (3) a firmware-faithful threshold optimizer that exhaustively evaluates the discrete (AP, RT) grid via a Rapid Trigger FSM simulator, returning the minimum-loss configuration and a Pareto frontier over detection rate and timing accuracy, replacing manual threshold tuning with per-user data-driven calibration. Together, these components form a practical recommendation workflow from raw HITS telemetry to interpretable parameter recommendations for click sensitivity tuning.

Contents

1. Introduction

Human-Computer Interaction (HCI) in high-performance computing and competitive esports places stringent demands on input peripherals. In environments where users execute several hundred Actions Per Minute (APM), actuation latency and threshold stability can affect both task performance and musculoskeletal load [22, 31].

1.1 The Evolution of Continuous Sensing Hardware

Peripheral hardware has shifted from binary mechanical switches toward continuous sensing interfaces. In HCI, threshold selection and activation logic have long been recognized as key determinants of usability and speed [17, 20]. Recent Hall-effect and inductive devices expose depth-dependent actuation and reset behavior to firmware, including Rapid Trigger mechanisms for dynamic re-actuation [6, 26, 36]. Commercial systems such as HITS further combine inductive sensing with configurable haptic feedback [9, 18, 19, 30]. Collectively, these developments increase control resolution but also increase dependence on well-calibrated software thresholds [3, 33, 34].

1.2 The Limitations of Static Actuation Thresholds

Despite these hardware leaps, the software interpreting this data remains limited. Current analog peripherals rely on static firmware thresholds that are agnostic to human biomechanical variance. Players exhibit unique control signatures, including micro-hesitations and varying velocity curves, which often clash with rigid hardware logic [17]. When static thresholds are set too aggressively, they fail to filter out physiological noise; when set too conservatively, they introduce unnecessary mechanical pre-travel. This friction results in suboptimal latency or a high frequency of unintended actuations during high-stress scenarios [20, 33].

1.3 Neuromotor Fatigue and Performance Decay

These hardware trends also intersect with user health. Higher APM in esports has been associated with repetitive strain injuries and tendinopathies [7, 14, 22]. Biomechanical studies report distinct fatigue patterns for keyboard and mouse interaction [16], and repetitive clicking can degrade aiming accuracy while increasing forearm muscle load [8]. High-APM genres are linked to elevated kinematic demand on wrist extensors [31], with measurable neuromotor adaptation during fatiguing tasks [12]. These findings motivate adaptive calibration that maintains responsiveness while reducing unintended actuation under fatigue.

1.4 Toward Adaptive Intent Detection

One longer-term direction is intent inference before full mechanical actuation. Prior work on surface electromyography (sEMG) and embedded TinyML suggests this is technically plausible [2, 15, 28, 35]. However, this thesis focuses on a narrower and directly evaluable scope: offline calibration from depth telemetry already produced by the peripheral. On-device inference is treated as future work (Section 7.3). Productization and UX integration (including the UX Exploration v2 flow and eventual GHub/GHabits integration) are likewise positioned as future work.

1.5 Thesis and Contributions

The central hypothesis of this thesis is that per-user variation in click telemetry is structured enough to support automatic labeling, statistical modeling, and parameter optimization beyond static default settings. The contributions are:

- A hybrid labeling pipeline for analog HID click intent that derives initial frame-level `MAKE_ONSET`, `CLICK_HOLD`, and `BREAK_ONSET` annotations from algorithmic criteria (geometry, speed, and acceleration) and then improves boundary quality through model-assisted iteration.
- A four-class temporal sequence classifier with two model paths: a shallow path (Random Forest, Gradient Boosted Trees) operating on handcrafted sliding-window features for sub-second per-session training, and a deep path (dilated residual FCN) operating on raw telemetry with a 3,060-frame (≈3 s) learned receptive field.
- A firmware-faithful threshold optimizer that exhaustively evaluates the discrete (AP, RT) grid

via a Rapid Trigger FSM simulator, returning the minimum-loss configuration and a Pareto frontier over detection rate and timing accuracy, replacing manual threshold tuning with a data-driven trade-off curve.

While not a primary algorithmic contribution, the work also includes a small HCI-facing component: purpose-built minigames that recreate common competitive-gaming input scenarios, providing a structured environment for users to generate calibration telemetry and receive a hardware recommendation. The recommendation outputs are structured for interpretability (trade-off views and comparative candidate points), so users can inspect why a suggested AP/RT point is proposed.

2. Background

To understand the design and implementation of an adaptive actuation system, it is necessary to establish the technical foundations of continuous sensing hardware, the mathematical logic of modern trigger parameters, and the biomechanical constraints of the human hand.

2.1 Continuous Depth-Sensing Mechanisms

Traditional mechanical switches operate on a binary principle: a circuit is either open or closed based on a physical contact point. In contrast, continuous depth-sensing switches measure the absolute position of the stem throughout its entire travel distance.

2.1.1 Hall-Effect and Magnetic Sensing. Hall-effect sensors utilize a permanent magnet attached to the switch stem and a sensor on the Printed Circuit Board (PCB). As the magnet approaches the sensor, the Hall voltage varies proportionally to the magnetic field strength [26, 36]. This allows the firmware to map voltage levels to precise displacement values in millimeters. Compared to classic contact switches, this technology reduces dependence on debounce filtering and enables lower-latency signal processing.

2.1.2 Haptic Inductive Trigger System (HITS). Refined in the 2026 Logitech SUPERSTRIKE series, the Haptic Inductive Trigger System (HITS) replaces magnets with an inductive coil architecture [19, 30]. By measuring changes in electromagnetic induction caused by the proximity of a metallic puck within the trigger, HITS provides higher resolution and lower susceptibility to external magnetic interference compared to Hall-effect sensors. Furthermore, HITS integrates localized haptic motors to simulate tactile "bumps" or "clicks" at software-defined depths, decoupling the physical feel of the switch from its electrical actuation point [9].

2.2 Control Parameters: AP and RT

The transition to continuous sensing introduced two primary variables that define the "feel" and speed of an input device.

2.2.1 Actuation Point (AP). The Actuation Point (AP) is the specific depth (d) at which a button-press event is registered. In traditional switches, this is fixed (e.g., 2.0mm). In analog systems, AP is a variable $d_{act} \in [0.1, 4.0]$mm. While these parameters are characteristically expressed as spatial displacement for keyboards, in the context of mice, a specific actuation depth inherently corresponds to a specific physical force applied onto the keyplates. Lowering d_{act} reduces mechanical pre-travel (or actuation force), allowing for faster response times, but increases the risk of accidental actuations caused by resting finger weight.

2.2.2 Rapid Trigger (RT). Rapid Trigger (RT) eliminates the fixed reset point found in mechanical switches. In a standard switch, the button must travel back up past a specific reset point before it can be pressed again. RT logic instead monitors the direction of travel. As soon as the stem moves upward by a specific threshold (Δ_{up}), the input resets. Conversely, as soon as it moves back down by Δ_{down}, it reactuates [6, 36]. RT mechanisms differ primarily in their reference boundaries: dynamic RT algorithms track the deepest and highest positions continuously along the entire travel path, only terminating this dynamic tracking when the displacement reaches absolute zero (a complete release). By contrast, other RT implementations restrict this dynamic tracking exclusively to the domain below the Actuation Point (AP), instantly terminating the rapid trigger state if the stem ascends

past the initial AP threshold. Both methods allow for rapid repeated clicking, which is critical in high-APM genres.

2.3 Biomechanical Variance and Neuromotor Fatigue

The efficiency of AP and RT is limited by both hardware imperfections and the physiological state of the user. Before examining human motor control, it is important to acknowledge that the raw sensor data is inherently noisy due to analog-to-digital converter (ADC) polling jitter, quantization noise, and non-linearities at the physical extremes of switch travel (top-out and bottom-out crush). However, even assuming a perfectly linear sensor, human motor control is subject to "signal-dependent noise," where the variance of movement increases with the force or speed of the action.

2.3.1 Resting Pressure and Tremors. Even at rest, users exert varying degrees of force on a peripheral. Factors such as cognitive stress or caffeine intake can cause micro-tremors-high-frequency, low-amplitude oscillations in displacement telemetry [13]. If the AP is set to 0.1mm and the user's tremor amplitude is 0.15mm, the system will register "ghost" inputs.

2.3.2 Neuromotor Fatigue Signatures. Extended sessions lead to fatigue in the *flexor digitorum superficialis* and *extensor digitorum* muscles [8, 16]. Fatigue manifests in two measurable telemetry changes:

1. **Velocity Decay:** The "snap-back" velocity (the speed at which a finger releases a key) decreases as the extensor muscles tire [32].
2. **Resting Weight Shift:** As the flexor muscles lose tonicity, the resting weight of the finger often increases, leading to deeper "resting" displacement in the analog sensor [12, 14].

2.4 Constraints of Offline Calibration (Scope Context)

This thesis focuses on offline calibration from depth telemetry. While continuous telemetry at high polling rates could theoretically be processed via on-device inference using quantized INT8 models on peripheral MCUs [15], hardware-in-the-loop deployment requires firmware-level access and strict latency budgets [23]. Consequently, on-device Edge ML is treated as future work, and all calibration pipelines in this study operate offline.

2.5 Signal Processing and Optimisation Foundations

The adaptive calibration pipeline draws on several established signal processing and machine learning methods. This section describes their mathematical properties; their problem-specific justifications appear in the Design chapter.

2.5.1 Savitzky-Golay Smoothing. Savitzky-Golay (SG) filtering fits a polynomial of degree p to each sliding window of W samples in a least-squares sense and evaluates the polynomial at the window centre [27]. Because the convolution coefficients are precomputed from the normal equations, the runtime cost is $O(n)$ regardless of window width, identical to a moving average. Crucially, the polynomial fit exactly reproduces moments up to degree p, so peak positions and edge inflection points in the signal are not distorted—a property absent from symmetric Gaussian or box filters, which both shift extrema toward the window centre.

2.5.2 Hermite Cubic Interpolation: Akima and Makima. Akima interpolation constructs a piecewise cubic Hermite spline whose tangent at each knot is a weighted average of the four adjacent finite differences, with weights inversely proportional to the absolute difference between consecutive slopes [1]. This weighting isolates the tangent estimate from outlier slopes: a single anomalous interval does not propagate ringing across the spline, unlike a natural cubic spline whose tangents are coupled globally. Akima does not enforce monotonicity, so it can represent non-monotone smooth curves. Modified Akima (Makima) adds a half-mean-slope regularisation term to the weight formula, preventing zero-weight degeneracy when two adjacent slopes are equal and opposite—a degenerate case that arises frequently in quantized sensor signals.

2.5.3 Covariance Matrix Adaptation Evolution Strategy. To optimize physical hardware thresholds like AP and RT, we require black-box optimizers that handle continuous, correlated variables well. CMA-ES is a derivative-free evolutionary optimiser for continuous search spaces [11]. It maintains a multivariate Gaussian distribution over candidate solutions and, at each generation, moves the distribution mean toward the best-performing candidates and adapts the full covariance matrix to reflect the local curvature of the objective landscape. This covariance adaptation lets the algorithm exploit correlations between parameters (such as the interdependent relationship between optimum AP and RT points) and navigate elongated, ill-conditioned valleys that cause gradient-free methods with diagonal step sizes (e.g. Nelder-Mead or coordinate descent) to stall. CMA-ES converges in $O(n^2)$ function evaluations for an n-dimensional problem, far fewer than the $O(N^n)$ evaluations required by grid search over a width-N grid.

2.5.4 Ensemble Methods: Random Forests and Gradient Boosting. For the offline ML pipeline, ensemble methods are particularly suited for extracting features from sliding-window time-series data because they are robust to scale differences and irrelevant features. A Random Forest [5] builds an ensemble of decision trees, each trained on a bootstrapped subsample with a random subset of features considered at each split, and averages their posterior class probabilities. The double randomisation decorrelates tree errors, yielding ensemble variance lower than any individual tree without significantly increasing bias. Gradient Boosted Trees [10] instead fit trees sequentially: each tree is trained to correct the pseudo-residual of the current ensemble via gradient descent on a differentiable loss. This staged correction reduces bias relative to a Random Forest, at the cost of longer training time and greater sensitivity to hyperparameter tuning.

2.5.5 Dilated Fully Convolutional Networks. While ensemble methods excel with hand-crafted features, deep learning models can automatically extract hierarchical representations directly from raw, unaligned telemetry data. Fully Convolutional Networks (FCNs) adapted for time-series employ stacked layers of 1D dilated convolutions. Dilated convolutions expand the temporal receptive field exponentially without requiring pooling layers that destroy temporal resolution, making them well-suited for capturing long-range dependencies in continuous analog sensor streams [4].

3. Related Work

This chapter surveys prior work across four domains that the proposed system draws on: adaptive threshold methods for HID devices, temporal sequence labeling from biosignals, programmatic labeling and weak supervision, and dilated convolutional architectures for sequence modeling.

3.1 Adaptive Threshold Methods for HID Devices

The problem of optimizing actuation parameters for human input devices has a long history in HCI, though prior approaches have relied on fixed heuristics rather than per-user learned models.

Kim et al. [17] introduced *impact activation*, an alternative actuation scheme for mouse buttons in which a click is registered at the first contact force peak following a rapid finger descent, rather than at a fixed displacement threshold. Their evaluation showed faster reaction times and fewer double-click errors under time pressure. Importantly, impact activation still relies on a single fixed heuristic (the first force peak), making it unsuitable for users with atypical contact dynamics, and it does not adapt to within-session fatigue. This thesis learns thresholds from the user's own recorded telemetry rather than applying a globally fixed policy.

Trewin [33] proposed the *Dynamic Keyboard*, which automatically adjusts keyboard response parameters (minimum press duration, maximum bounce interval) to accommodate users with motor impairments, by observing an initial calibration sequence and adjusting parameters using rule-based adaptation. The Dynamic Keyboard targets binary on/off switch events and adjusts temporal debounce windows; it does not exploit continuous analog depth signals, and its adaptation rules are domain-expert heuristics rather than models trained from user data. The proposed system is

complementary in motivation—both aim to reduce unintended actuations—but operates at a different signal level (continuous depth at 1 kHz vs. binary event timing) and uses a data-driven model rather than hand-coded rules.

To the best of our knowledge, peer-reviewed literature has not yet reported a complete learning-based pipeline for automatic joint tuning of Rapid Trigger (RT) and Actuation Point (AP). Current commercial implementations [18, 36] primarily expose RT and AP as user-adjusted controls without closed-loop personalization. The firmware-faithful threshold optimizer in this thesis therefore positions RT/AP tuning as an explicitly optimized per-user objective rather than a manual configuration task.

3.2 Temporal Sequence Labeling in Biosignal Processing

Detecting click intent from continuous depth telemetry is an instance of temporal sequence labeling, a problem that has received substantial attention in the context of richer biosignals.

Surface electromyography (sEMG) is the closest related modality. Wang et al. [35] presented ReactEMG, which uses masked autoencoder pre-training on sEMG to achieve stable, low-latency intent detection from eight-channel surface electrode recordings. Antuvan and Masia [2] applied Linear Discriminant Analysis to simultaneous classification of wrist and finger movements from sEMG, demonstrating real-time throughput on embedded hardware. Both systems operate on multi-channel electrode signals that encode rich neuromuscular information before mechanical actuation occurs; their signal-to-noise advantage over single-sensor depth telemetry is substantial. The proposed system, by contrast, uses only the depth signal already present in the peripheral itself, requiring no additional sensing hardware. This imposes a harder classification problem—depth telemetry cannot observe pre-actuation motor intent—but removes the deployment barrier of wearable electrodes.

Keystroke dynamics literature [21, 29] models typing patterns for biometric authentication from binary key event timestamps. TypeFormer [29] applies a Transformer architecture to sequences of inter-key timings and achieves high verification accuracy on mobile devices. These methods treat the keystroke as an atomic event and model the temporal pattern between events; they do not model sub-millisecond intra-click dynamics. The proposed system operates at a finer granularity—frame-level classification at 1 kHz—which requires a different feature engineering approach and exposes a different class of failure modes (contact-bounce artefacts, quantization noise) absent from binary event streams.

3.3 Weak Supervision and Algorithmic Bootstrapping

The geometric auto-labeler introduced in Section 4.2 fundamentally differs from programmatic weak supervision frameworks such as Snorkel [25]. Snorkel defines *labeling functions*—heuristic rules, distant supervision signals, or domain-expert logic—that each assign noisy labels to unlabeled examples, and combines them via a generative model that learns the accuracy and correlations of each labeling function. The combined label distribution is then used as the training signal for a downstream discriminative model.

Instead of deploying the geometric auto-labeler as an automated weak supervision signal for model training, the proposed system employs it strictly as an algorithmic bootstrapping assistant for human-in-the-loop annotation. Purely algorithmic segments (i.e., those assigned by the auto-labeler but not verified) are explicitly stripped from the dataset during the training pipeline; the machine learning architectures are trained exclusively on user-verified and manually adjusted boundaries. This conservative data policy ensures high-fidelity ground truth and prevents the model from merely mirroring the systematic biases and plateau ambiguities of the underlying geometric heuristic. As a result, the proposed system acts as a strongly supervised learning pipeline accelerated by algorithmic pre-labeling, rather than a weakly supervised system relying on noisy heuristics. To account for natural human inconsistency in selecting the exact millisecond of an onset during this manual pro-

cess, the system employs asymmetric soft labels (described in Section 4.4) that encode physiological directional uncertainty into the verified targets, rather than treating boundary annotations as rigid, perfect temporal observations.

3.4 Dilated Convolutions for Sequence Modeling

The OfflineFCN architecture adapts the dilated causal convolution structure introduced by van den Oord et al. in WaveNet [24] for raw audio generation. WaveNet stacks dilated causal convolutional layers with exponentially increasing dilation rates to achieve a large receptive field with manageable parameter counts. The dilation schedule $d_i = 2^i$ allows the stack to cover $\sum 2^i$ steps of context with $O(\log N)$ layers rather than the $O(N)$ layers that a fixed-dilation architecture would require.

The OfflineFCN departs from WaveNet in one critical dimension: it uses symmetric (non-causal) padding rather than causal (left-only) padding. This doubles the effective receptive field per layer for the same kernel size, since context is drawn from both directions—enabling the 3,060-frame ($\approx$3 s at 1 kHz) receptive field described in Section 4.4 to be achieved with only 8 blocks. The trade-off is that the resulting model cannot be deployed for streaming inference, since each output depends on future samples. This is appropriate for the offline analysis mode but excludes the model from the on-device path described as future work.

4. Design

This chapter details the architecture and algorithmic design of the proposed adaptive actuation system.

4.1 System Architecture

The system utilizes a multi-layered software and hardware stack designed to bridge high-frequency physical telemetry with a dynamic optimization engine. At the hardware layer, a modified PRO X2 Superstrike mouse streams continuous 120-level analog depth values at 1 kHz. This bypasses typical firmware constraints that generally limit reporting to mere 4-bit precision at 65 Hz and block reading after haptic events. The hardware communicates with the host operating system via an OS-level driver bridge that interfaces high-level prototyping environments with production-grade HID protocols. Sitting above this is a background sensor service that orchestrates and synchronizes the analog telemetry with OS events and other biometrics. Finally, a front-end orchestration client provides the user dashboard, manages recording state, and drives the downstream machine learning pipelines. The overall component topology is shown in Figure 4.1.

4.2 Data Labeling Methodology

Raw analog signals are inherently noisy. Before any label placement, the depth telemetry is smoothed using Modified Akima (MAKIMA) interpolation. MAKIMA was specifically chosen because it preserves signal monotonicity and avoids the severe overshoots characteristic of standard cubic splines, which is critical for accurate boundary detection on a signal with sharp transitions.

4.2.1 Algorithmic Bootstrapping. To alleviate the bottleneck of manual annotation at scale, an automated heuristic pipeline pre-populates boundary candidates before human review. Detection anchors on the OS-reported digital key event to identify the approximate click window, then walks the analog signal to locate the true biomechanical boundary:

- **Make Intention:** Starting from the OS make event, the algorithm walks backwards in time to find the first non-zero analog depth, or the immediately preceding local trough or valley in the depth signal. A hysteresis constraint prevents spurious detections from sensor noise near rest position.
- **Break Intention:** Detected at the first zero-crossing of the analog velocity (i.e., the first local peak in depth after the maximum), which typically occurs earlier than the OS-reported key release and more accurately reflects the physical start of the finger lift.

This heuristic correctly handles the vast majority of clean, well-separated clicks, providing an accurate starting point for human reviewers to correct

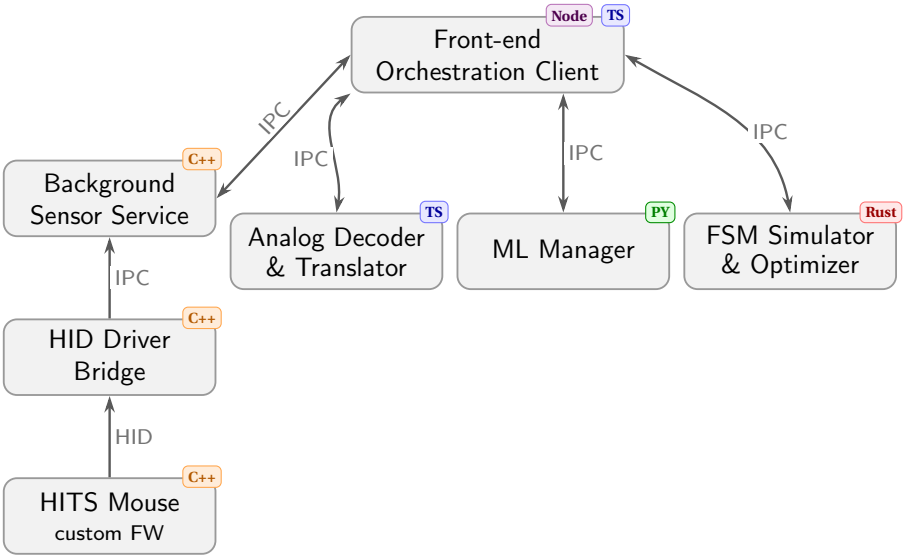

Figure 4.1: System architecture. Telemetry flows upward from a HITS-enabled mouse with custom firmware through a C++ HID driver bridge and a C++ background sensor service via IPC to the front-end orchestration client (Node/Electron + TypeScript). Three independent out-of-process backend services communicate bidirectionally with the client via IPC: an analog decoder and translator (Python + TypeScript), a machine learning manager (Python), and a hardware FSM simulator and optimizer (Rust).

rather than annotate from scratch.

4.2.2 Iterative Human-in-the-Loop Annotation. Annotation followed an iterative three-phase strategy designed to maximize label quality while minimizing annotation effort:

1. **Algorithmic bootstrap.** The heuristic pipeline above generated initial boundary candidates for all recordings. Annotators reviewed these within a custom data visualization and labeling interface integrated directly into the front-end orchestration client. The interface renders the raw 1 kHz analog depth signal alongside its derived velocity, and allows annotators to drag start and end boundary markers independently for the left and right button channels. Visual smoothing was applied on a per-recording basis to assist in distinguishing genuine kinetic events from micro-tremors. Annotators focused strictly on biomechanical intent, preserving deliberate presses including those not registered by the OS, while discarding involuntary micro-tremors and resting-weight drift. Approximately 50% of the dataset was verified and corrected in this phase.
2. **Model-assisted labeling.** Once the manually verified corpus was sufficiently large, a preliminary model was trained and used to autolabel the remainder of the dataset. Segments with a model confidence of 95% or above were accepted automatically, covering approximately 60–80% of the remaining unlabeled recordings.
3. **Mixed review.** The remaining recordings — primarily those containing clicks not registered by the OS, which the heuristic cannot anchor on — were processed by a combination of the algorithmic heuristic and direct human annotation. A final manual review pass was conducted over all automatically labeled segments to validate quality before inclusion in the training corpus.

The full extent and breakdown of the resulting annotated corpus is described in Section 5.1.

4.3 Data Preparation Pipeline

Raw recordings and their human-verified labels are transformed into normalized arrays shared by all three model architectures. Several design decisions govern this pipeline. First, onset boundary zones are widened by a small tolerance radius to absorb natural annotation imprecision, but are adaptively clamped against adjacent segment boundaries to prevent cross-segment label contamination. Second, derivative channels (velocity, acceleration, jerk, and resting-drift) are derived using zero-phase filtering to avoid introducing causal shift artifacts. Third, idle background periods are trimmed by an activity mask to reduce the dominance of background frames without discarding legitimate negative examples. Fourth, dataset splits are performed at the participant level so no individual's recordings span both training and evaluation. Finally, normalization statistics are fitted on the training partition only, and statistics for kinematic channels are computed over active frames exclusively to prevent background mass from compressing the feature scale. Concrete implementation details—feature extraction windows, normalization constants, and weighting formulas—are described in Section 5.3.

4.4 Click-Intent Sequence Modeling

The click-intent problem is formulated as a per-frame four-class temporal segmentation task over continuous 1 kHz sensor streams. Each frame is assigned one of four mutually exclusive labels: Background, Make Onset, Click Hold, and Break Onset. Three model architectures are supported, positioned along a speed–accuracy trade-off curve suited to the realities of on-device, per-session deployment.

4.4.1 Random Forest. The Random Forest classifier operates on hand-crafted sliding-window features extracted from a local neighbourhood around each frame. It is the primary production model. Its main strength is training speed: a full session retrains in a few minutes on a consumer CPU. Feature importance scores are directly interpretable, exposing which kinematic cues drive the model's decisions. The main weakness is a bounded temporal horizon limited to a few tens of milliseconds of context, which is sufficient for most click onset detection but may struggle on ambiguous edge cases where the intent signal unfolds over a longer transition. This architecture also fails to capture long-range dependencies and long context of user behavior. The pipeline also requires domain knowledge to design and cannot adapt to latent signal structure that was not anticipated during feature engineering.

4.4.2 Gradient Boosted Trees. The Gradient Boosted Trees model shares the same feature extraction pipeline as the Random Forest but replaces the ensemble learner with sequential boosting, which iteratively focuses on the residuals of prior trees. Its key advantage is support for custom loss functions: a focal loss variant down-weights easy background frames, and a Gaussian soft-label objective encodes labeling uncertainty at onset boundaries—both of which reduce over-confidence on the majority background class. Early stopping against a held-out validation metric limits overfitting. The weaknesses are slower training than the Random Forest, greater hyperparameter sensitivity, and the same fundamental constraint of a short-context hand-crafted feature representation.

A distinct design choice for the GBT path is the use of **asymmetric soft labels**. Standard hard labels treat each onset frame as a binary ground truth, but human annotators naturally place boundaries with some imprecision, and the physiological consequences of timing errors are asymmetric. For the Make Onset, detecting slightly *early* is acceptable (the finger is already moving), whereas a *late* detection directly increases perceived input latency. Conversely, for the Break Onset, a slightly *late* detection is tolerable (the click has already been registered), whereas an *early* detection prematurely resets the input state. The soft label generator therefore uses a split-normal Gaussian centred on each onset frame: a wide left standard deviation and narrow right standard deviation for Make, and narrow left and wide right for Break. This encodes directional uncertainty directly into the training target, preventing the model from being equally penalised for both error directions.

4.4.3 Fully Convolutional Network. The dilated residual FCN operates directly on raw normalized sensor streams without manual feature engineering. Stacked dilated convolutions grow the receptive field exponentially: with eight dilation stages and a kernel size of 7, the effective receptive field spans approximately 3 060 frames (≈3 s at 1 kHz), allowing the model to condition each frame's prediction on long-range temporal context—pre-click muscle tension ramp-ups and post-click rebound dynamics that fall well outside any practical sliding window. The model is trained end-to-end on a three-channel Gaussian heatmap target, producing soft onset probability curves rather than hard class assignments. The trade-off is higher computational cost: the FCN requires a GPU or significantly longer CPU training time, is less interpretable than tree-based models, and benefits more strongly from larger datasets. It is therefore positioned as the high-accuracy architecture for offline post-processing, while the Random Forest handles latency-sensitive or resource-constrained deployment.

4.5 From Classification to Segmentation

The per-frame classifier produces a four-class probability array over the full recording. Converting these continuous probability outputs into discrete (start, end) click segments requires three sequential design choices: identifying candidate onset regions, electing a single representative frame from each region, and pairing each Make onset region with its corresponding Break onset region.

Candidate run detection. Two separate probability channels are extracted from the classifier output: the Make Onset probability and the Break Onset probability. Contiguous regions above a configurable threshold in each channel are identified as candidate runs. For Make runs, a Schmitt-trigger hysteresis is applied: a run opens when the Make probability exceeds the make threshold and remains open until it falls below half that threshold. This bridging behavior prevents a single physical press—whose probability hump can dip below the threshold due to quantization noise or softmax saturation—from being fragmented into multiple spurious candidates. Break runs are detected with a simpler thresholding scheme.

Onset frame election. Within each candidate run, a single frame must be elected as the representative onset. Three strategies are evaluated: **peak** selects the argmax frame; **plateau median** identifies the set of frames within 0.5% of the peak probability (the plateau) and returns the median frame index, falling back to the argmax when the plateau collapses to a single point; **steepest rise/fall** selects the frame of maximum gradient, corresponding to the inflection point of the probability curve and providing an estimate that is independent of plateau saturation effects. The optimal strategy and its associated make and break threshold values are selected jointly via a Bayesian hyperparameter sweep (Optuna) over the validation set, optimizing the Intersection over Union (IoU) between predicted and ground-truth click segments.

Make–Break pairing. Given a set of elected Make onset frames and Break onset frames, a final matching step assigns each Make to exactly one Break. Three pairing strategies are available. The simplest is a **greedy first-fit** approach: each Make is paired with the first eligible Break that falls within the Make run's search window. This serves as a baseline. The second strategy formulates the assignment as a **linear sum assignment problem** (Hungarian algorithm), constructing a cost matrix whose entry (i, j) is the negative geometric mean of the Make and Break confidence scores; the global minimum-cost assignment is then solved exactly in $O(n^3)$ time, ensuring no Break is claimed by more than one Make. The third strategy extends the Hungarian approach with a **trained pairing scorer**: a logistic regression classifier, fit on positive and negative (Make, Break) pairs extracted from the training set, replaces the raw confidence product as the cost signal. The scorer uses six features per candidate pair—Make confidence, Break confidence, log-duration between the elected frames, Make run width, Break run width, and the number of competing Break runs within the Make's search window—and is trained with balanced class weights to handle the natural scarcity of positive (correct) pairs. The

optimal pairing strategy is selected by evaluating all three on the validation set IoU, and the chosen strategy is stored alongside the model weights so inference uses identical pairing logic.

4.6 Parameter Optimization

After click-intent boundaries are extracted, the optimizer estimates the hardware threshold pair that best reproduces those boundaries under firmware-faithful trigger simulation.

4.6.1 Firmware State Machine Simulation. Each candidate threshold pair is evaluated with a frame-accurate simulation of firmware trigger logic.

In **simple-threshold mode**, the simulator is a 2-state machine: press is emitted when the make signal crosses the make threshold upward, and release is emitted when the break signal crosses the break threshold downward.

In **Rapid Trigger mode**, the simulator is a 3-state machine (Idle → Pressed → RT-Released) with rolling extrema. After actuation, release is emitted when depth falls by at least RT sensitivity below the running peak. From RT-Released, re-actuation occurs when depth rises by at least RT sensitivity above the running valley. The state resets to Idle only when the signal returns to zero, matching firmware reset semantics.

To preserve firmware behavior while keeping search efficient, signals are quantized before evaluation and searched on a discrete, hardware-relevant threshold grid.

4.6.2 Search Space and Objective. Optimization is discrete and exhaustive over the candidate threshold grid, yielding deterministic and reproducible solutions at exposed hardware resolution.

For each candidate, simulated click intervals are matched to labeled intervals using **overlap-constrained monotonic matching**. A label is matched only if at least one simulated interval overlaps that label interval. If multiple simulated intervals overlap the same label (oscillation/chatter), they are collapsed into one canonical matched interval using earliest make and latest break.

Let

$$\begin{aligned} N &= \text{number of labeled clicks,} \\ TP &= \text{number of matched labels,} \\ FN &= N - TP, \\ FP &= \text{number of unmatched simulated intervals.} \end{aligned}$$

For matched pair i, define make and break boundary errors:

$$e_i^{(m)} = t_{i,\text{sim}}^{(m)} - t_{i,\text{label}}^{(m)},$$
$$e_i^{(b)} = t_{i,\text{sim}}^{(b)} - t_{i,\text{label}}^{(b)}.$$

Early errors are asymmetrically penalized with multiplier $\kappa \geq 1$:

$$\phi_\kappa(e) = |e|\,(1 + (\kappa - 1)\mathbf{1}[e < 0])\,.$$

Per-match timing distance is

$$d_i = w_{\text{make}}\,\phi_\kappa\left(e_i^{(m)}\right) + (1 - w_{\text{make}})\,\phi_\kappa\left(e_i^{(b)}\right),$$

where $w_{\text{make}} \in [0,1]$ controls make-vs-break priority.

Detection penalties are parameterized by base ms-equivalent detection cost c_{det} and recall/precision bias β:

$$c_{\text{miss}} = c_{\text{det}} e^{\beta}, \qquad c_{\text{fp}} = c_{\text{det}} e^{-\beta}.$$

The final loss is

$$\mathscr{L} = \begin{cases} \infty, & TP = 0, \\ \dfrac{1}{TP}\displaystyle\sum_{i=1}^{TP} d_i + \dfrac{c_{\text{miss}} FN + c_{\text{fp}} FP}{N}, & TP > 0. \end{cases}$$

This yields interpretable controls: w_{make} sets make-vs-break timing emphasis, κ controls early-click severity, and (c_{det}, β) control overall detection strictness and recall-precision preference.

4.6.3 Pareto Frontier and Output. For each feature-pair configuration, the optimizer returns all evaluated candidates, the minimum-loss operating point, and a Pareto frontier over {error rate, average click distance} for interactive inspection.

Detailed serialization choices (grid layout, sen-

tinels, and per-combination diagnostics) are implementation concerns and are described in Section 5.5.

5. Implementation

This chapter describes the software and hardware implementation of the system designed in Chapter 4.

5.1 Data Acquisition Campaign

The development and training of the machine learning backend required the construction of a robust ground-truth dataset spanning diverse ergonomic and biomechanical profiles. Data was collected from 69 participants via two separate campaigns: a controlled task dataset, including both gamers and non-gamers, in-house, and a free-play dataset at the PolyLAN event. The two datasets were collected under different conditions to capture both structured and unstructured click dynamics.

5.1.1 Controlled Task Dataset (BIC Event). The first deployment cohort ($N = 26$) comprised a mix of competitive gamers to casual novices playing *Counter-Strike 2* (CS2). The protocol ran for approximately 40 minutes per user and was strictly structured to isolate specific click dynamics:

- **Configuration A/B Testing:** Following a uniform warmup phase, users played two 15-minute game blocks. The device hardware profile was hot-swapped halfway through the session, alternating between an aggressive configuration (AP at 1, RT at 1) and a highly conservative configuration (AP at 5, zero Rapid Trigger). The presentation order was randomized to mitigate sequence bias.
- **Weapon Rotations:** Within each 15-minute block, users were instructed to rotate their active weapon every five minutes. The arsenal was specifically chosen to evoke distinct motor patterns: the *USP-S* pistol for rapid individual tapping, the *M4A1-S* rifle for sustained tracking and spraying, and the *AWP* sniper rifle for slow, deliberate holding and release.

Qualitative user feedback regarding the switch feel was collected asynchronously after the sessions via standardized surveying.

5.1.2 Free-Play Dataset (PolyLAN Event). The secondary dataset ($N = 43$) focused on unstructured free-play in a high-density LAN environment. Participants engaged in approximately 40-minute play sessions in their preferred title (*CS2*, *Valorant*, or *League of Legends*). Similar to the controlled dataset, the hardware configurations were swapped from conservative to aggressive baseline profiles near the 20-minute mark to capture a wider range of organic responses. FPS players produced predominantly left-button click profiles consistent with primary-fire mechanics; *League of Legends* players generated a contrasting signature of high-frequency right-click interactions, characteristic of point-and-click action commands.

The aggregated dataset generated from these two campaigns ultimately produced hundreds of hours of raw 1 kHz telemetry, demanding the variance entropy scoring (detailed in Section 4) to isolate periods of genuine engagement before entering the classification pipelines.

5.1.3 Annotated Dataset Summary. Following manual annotation and variance-based trimming, the final curated dataset comprises 132 labeled recording files from 39 FPS-player participants, totaling approximately 12,196 seconds (~3.4 hours) of verified active telemetry. Left-channel annotations account for 39,088 labeled click cycles and right-channel annotations for 6,645 cycles, reflecting the predominance of primary-fire left-button interactions in FPS gameplay. The dataset was partitioned into training, validation, and test splits performed at the participant level to enforce strict subject isolation. We decided to hold out the League of Legends participants from the scope of this thesis, as their click dynamics were not representative of the FPS-focused design goals.

5.2 Frontend and Telemetry Stack

The user-facing layer provides the orchestration dashboard, recording states, and test minigames. Telemetry ingestion is handled out-of-process by a dedicated background sensor service, ensuring that

1 kHz hardware polling never stalls the frontend workflows.

5.2.1 Analog Decoder and Translator. Raw telemetry is stored as a sparse binary event stream capturing only state-change transitions. A dedicated decoder parses this format and resamples the event sequence to a uniform 1 ms grid using Modified Akima (MAKIMA) interpolation, which preserves the monotonic shape of the analog depth signal without the overshoot characteristic of standard cubic splines. Short resting gaps receive synthetic zero-value anchors before interpolation to prevent overshoot across idle periods. The output per frame consists of the quantized analog depth on a 0–120 scale, a digital press bit for each button, and a wall-clock timestamp. A client-side signal processing layer subsequently derives the kinematic channels—velocity, acceleration, jerk, and resting drift—from the interpolated depth, making them available for visualization and downstream annotation.

5.2.2 Labeling Interface. The annotation interface renders the decoded 1 kHz analog depth signal for both buttons alongside the derived kinematic channels, enabling annotators to judge biomechanical intent from the full kinematic context rather than raw depth alone. MAKIMA interpolation is applied at twice the native sampling rate on the client side to smooth the velocity overlay for visual clarity, without introducing causal shift. Annotators interact with segment boundaries via draggable handles: the start and end boundaries of each labeled region can be adjusted independently per channel, whole segments can be repositioned by body-drag, and new segments are created by dragging in unlabeled space. Visual overlays distinguish manual annotations from algorithmically generated candidates, and highlight model-generated suggestions alongside their associated confidence scores.

Annotation followed the three-phase iterative strategy described in Section 4.2. An algorithmic bootstrapping heuristic pre-populates boundary candidates to seed manual review. For the Make boundary, the algorithm walks the analog signal backward from the OS-reported key-down event, stopping at the first non-zero depth sample or the nearest preceding local trough to anchor the biomechanical press onset before the firmware actuation threshold was crossed. For the Break boundary, it identifies the first zero-crossing of the analog velocity after the press peak—the point at which the finger begins lifting—which consistently precedes the OS key-up event and more accurately reflects physical intent.

This heuristic performs well on clean, well-separated clicks but has systematic failure modes that motivate the full annotation pipeline. It relies entirely on the OS digital event as an anchor, so clicks that were biomechanically executed but not registered by the firmware—because the actuation threshold was set deeper than the finger reached—are entirely invisible to it. It is also sensitive to sensor noise near the rest position, where microtremors can produce spurious troughs that attract the backward walk and misplace the Make boundary. These failure modes—unregistered presses and noise-induced misplacements—are precisely the edge cases most informative for training a robust intent model, and are the primary reason human review remained indispensable throughout the annotation process.

5.3 Machine Learning Backend

The machine learning pipeline is encapsulated in a dedicated Python intent-detection backend built with scikit-learn, LightGBM, and PyTorch.

5.3.1 Loading and Frame Extraction. Each recording file is paired with a JSON label sidecar validated at load time. The 120-level analog depth stream is normalised to $[0, 1]$ by dividing by 120, and binary digital press signals are cast to $\{0.0, 1.0\}$. Only segments annotated as `source=manual` are ingested; algorithmically generated segments are discarded. Each manually annotated click segment maps to three frame-level classes: Make Onset, Click Hold, and Break Onset. Onset zones extend ± 4 ms from each boundary marker to absorb natural annotation imprecision—a tolerance calibrated empirically to preserve signal sharpness while remaining robust to small annotator variability—and are clamped against adjacent segment boundaries

to prevent cross-segment label contamination. A mtime-keyed in-memory cache prevents redundant re-parsing of unchanged files.

5.3.2 Preprocessing and Signal Augmentation. The analog channels optionally receive temporal smoothing (moving average, Gaussian, Savitzky-Golay, median, or EMA). Kinematic derivative channels—velocity, acceleration, jerk, and their absolute-value variants—are computed via zero-phase Akima-spline upsampling followed by centered finite differences and downsampling, eliminating causal shift. A resting-drift channel is derived as the difference of a fast and a slow exponential moving average, both applied zero-phase, capturing low-frequency postural drift independently of click dynamics. Continuous channels are optionally decimated using a zero-phase Chebyshev IIR low-pass filter before striding; discrete digital channels are strided only. A rolling-window activity mask based on configurable amplitude and variance thresholds trims inactive interior gaps, retaining short margins around each boundary and all labeled frames, and splits the recording into independent active sub-sequences.

5.3.3 Splitting, Normalization, and Sample Weighting. Sub-sequences are partitioned into train, validation, and test at the sequence level. The preferred strategy is participant-level k-fold cross-validation: sequences are grouped by `user_id` (extracted from the file path via `p\d+` regex), each fold holds out exactly one participant group, and sequences without a `user_id` always go to training. Z-score normalization is fitted on the training partition only. For kinematic channels (velocity, acceleration, jerk, drift) the standard deviation is computed over non-background frames exclusively to prevent the background mass from compressing the feature scale. Binary digital and constant per-recording configuration channels are excluded from normalization.

Click-balanced per-frame sample weights equalize participant contributions. For participant u with C_u click events and b_u background frames, the weight of a frame belonging to click event e spanning $|F_e|$ frames is $(1/C_u)/|F_e|$, and the weight of a background frame is $1/b_u$. Weights are then rescaled per class to preserve the click/background mass ratio, and capped at a maximum ratio of 10 via iterative convergence. The Random Forest additionally applies inverse-frequency class weights scaled by configurable per-class multipliers, providing an independent knob for the cost of each misclassification type.

5.3.4 Feature Extraction and Model Ingestion. For the Random Forest and Gradient Boosted Trees, sliding-window feature extraction produces one tabular vector per frame. Each window is centered on the target frame and computes a fixed bank of per-channel statistics: window mean, first-to-last slope, peak, velocity at the centre frame, time-to-peak, minimum, and time-to-trough. A set of cross-channel features is always appended: left/right analog correlation, whether each digital channel fires during an analog rise, and the maximum analog value across both switches. The feature set grows by one additional per-channel block for each derived channel enabled (e.g. velocity, acceleration, jerk, drift). An extended feature bank is also available, adding higher-order statistics such as zero-crossing rate, TKEO energy, and kurtosis. Optional context probes append raw or locally averaged analog samples at configurable temporal offsets (e.g. ±50 ms, ±100 ms).

The FCN bypasses tabular extraction and receives raw $[C, T]$ tensors directly. The architecture consists of an input projection convolution, eight stacked dilated residual blocks each containing two convolutional layers, and a per-frame output classifier—18 convolutional layers in total. The dilation factor doubles at each successive block $(1, 2, 4, \ldots, 128)$, producing an effective receptive field of

$$RF = 2(k-1)(2^D - 1)$$

which evaluates to 3 060 frames (≈3 s at 1 kHz) with kernel size $k = 7$ and $D = 8$ dilation stages. Each block applies symmetric non-causal padding—equal context on both sides of each convolution—so the model introduces no temporal shift in its predictions. A ReLU activation and dropout are applied between the two convolutional layers of each block; the skip connection within each residual

block is an identity map throughout, as all blocks share the same hidden channel dimension. Training labels are converted to a 3-channel heatmap: channels 0 and 1 carry Gaussian probability bumps (σ = 8 frames) centred on each Make and Break onset frame respectively, and channel 2 carries a binary hold mask. A focal binary cross-entropy loss is applied per channel per frame, with per-frame sample weights scaling the loss directly. When sequence-window training is used, an 8-frame zero-weight gap is inserted between recordings in the concatenated hypersequence so no sliding window can span a boundary.

5.3.5 Gradient Boosted Trees. The Gradient Boosted Trees model shares the sliding-window tabular feature extraction pipeline described above; every per-frame feature vector is identical to the one supplied to the Random Forest. The distinguishing design concerns are the training objective and the label targets.

At onset boundaries, hard one-hot labels are replaced by asymmetric soft targets. A split-normal Gaussian is centred on each annotated onset frame, with independent standard deviations on either side: for Make Onset, the left standard deviation (frames preceding the onset) is wider than the right, encoding that detecting slightly early is less harmful than a late detection; for Break Onset the asymmetry is reversed. Soft target vectors are normalized to sum to 1.0 per frame before training.

Two training objectives are supported depending on whether soft or hard labels are active. With hard labels, a focal cross-entropy variant suppresses the gradient contribution of easy background frames, reducing the dominance of the majority class. With soft labels, a custom gradient and Hessian objective replaces the standard multiclass cross-entropy to align with the continuous target distribution. In both cases, early stopping is evaluated on hard-label accuracy over the held-out validation partition, keeping the stopping signal interpretable and comparable across configurations.

5.4 From Classification to Segmentation

The per-frame classifier produces a probability array over all four classes for the full recording. Converting these continuous outputs into discrete (start, end) click intervals requires three sequential stages: identifying candidate onset regions, electing a single representative frame from each region, and pairing each Make onset with a corresponding Break onset.

5.4.1 Candidate Run Detection. The Make probability channel is processed with a Schmitt-trigger hysteresis: a candidate run opens when the probability exceeds a configurable entry threshold and remains open until it falls to half that value. This two-threshold bridging behavior prevents a single physical press—whose probability hump can dip transiently below the entry threshold due to softmax saturation or quantization noise—from being fragmented into multiple spurious candidates. Break runs are detected with a simpler single-threshold scheme, as the break probability signal is generally less susceptible to mid-run fluctuations.

5.4.2 Onset Frame Election. Within each candidate run, a single frame must be elected as the representative onset. Three strategies are implemented and evaluated jointly with the threshold parameters during hyperparameter selection:

- **Peak:** the frame with the highest probability within the run (argmax). Simple and effective when probability humps are narrow and well-localised.
- **Plateau median:** the median index among all frames whose probability lies within 0.5% of the run peak. Falls back to the argmax when no plateau exists. More robust than the peak strategy when the probability curve saturates and a large contiguous plateau forms near 1.
- **Steepest rise/fall:** the frame of maximum probability gradient—the inflection point of the probability curve. This estimate is insensitive to plateau saturation and most responsive to the sharpest temporal edge of the onset hump.

5.4.3 Make–Break Pairing. Given the elected Make and Break onset frames, a matching step assigns each Make to exactly one Break. Three strategies of increasing sophistication are evaluated:

- **Greedy first-fit** (baseline): each Make is paired with the first eligible Break that falls within the

Make run's search window.

- **Hungarian assignment:** the globally optimal one-to-one matching, solved via the linear sum assignment algorithm. The cost of pairing Make i with Break j is the negative product of their confidence scores, penalized by a term proportional to the temporal distance between the elected frames normalized by the search window, discouraging implausibly long-duration pairings.
- **Trained pairing scorer:** a logistic regression classifier fit on positive and negative (Make, Break) pairs extracted from the training set replaces the raw confidence product as the cost signal in the Hungarian framework. The scorer uses six features per candidate pair: Make confidence, Break confidence, log inter-onset duration, Make run width, Break run width, and the number of competing Break runs within the Make search window. Training uses balanced class weights to compensate for the natural scarcity of positive pairs.

5.4.4 Hyperparameter Selection via Bayesian Sweep.

The optimal combination of election strategy, entry threshold, and exit threshold is determined by maximizing the segmentation IoU between predicted and ground-truth click intervals on the held-out validation set. A Bayesian search using a Tree-structured Parzen Estimator (TPE) sampler is employed, backed by a persistent SQLite store that allows sweeps to be paused and resumed across sessions. When the Cartesian product of the discrete search space falls at or below 200 candidate combinations, the sampler automatically transitions to exhaustive grid enumeration to guarantee full coverage. The elected strategy, pairing strategy, and threshold values are stored alongside the trained model weights so that inference uses the identical configuration selected during validation.

5.5 Optimization Pipeline Integration

The parameter optimizer is implemented as a standalone Rust service that communicates with the front-end client via named-pipe IPC. It receives preprocessed analog signals and labeled click regions from the ML manager, performs an exhaustive discrete search over candidate thresholds, and returns both best-solution and analysis artifacts as JSON outputs.

The core of the service is a frame-accurate simulation of firmware trigger logic, illustrated in Figures 5.1 and 5.2.

The deployed optimization path uses depth for both make and break channels, with candidate thresholds (T_m, T_r) where T_m is Actuation Point and T_r is Rapid Trigger sensitivity. In this path, the simulator runs a 3-state FSM (Idle → Pressed → RT-Released):

- Idle → Pressed if $m_i \geq T_m$ (emit make, initialize running peak),
- Pressed → RT-Released if $b_i \leq \text{peak} - T_r$ (emit break, initialize running valley),
- RT-Released → Pressed if $b_i \geq \text{valley} + T_r$ (emit make),
- RT-Released → Idle only when the signal returns to zero ($b_i \leq 0$ or valley ≤ 0), matching firmware reset behavior.

For non-depth break features, the backend also supports a simple 2-state threshold mode with candidate (T_m, T_b):

- unpressed → pressed when $m_i \geq T_m$,
- pressed → unpressed when $b_i \leq T_b$.

Signals are quantized before evaluation so search remains firmware-faithful and efficient. In the deployed depth-depth path, thresholds are evaluated exhaustively over the exposed 10-level depth grid.

For each threshold candidate, simulated intervals are aligned to labels using overlap-constrained monotonic matching. The service computes timing and detection metrics (matched, missed, false positives, make-early, break-early, and latency summaries), evaluates the objective described in Section 4.6, and stores per-candidate simulated intervals and aggregate statistics.

Output artifacts include:

- the complete set of evaluated candidates,
- the minimum-loss solution,
- a Pareto frontier over {error rate, average click distance},

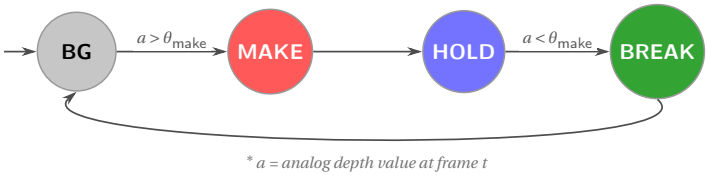

Figure 5.1: Simple threshold (no Rapid Trigger) FSM. A Make event fires when depth crosses above θ_{make}; a Break event fires when depth falls back below the same threshold. The four states correspond directly to the four classifier output classes.

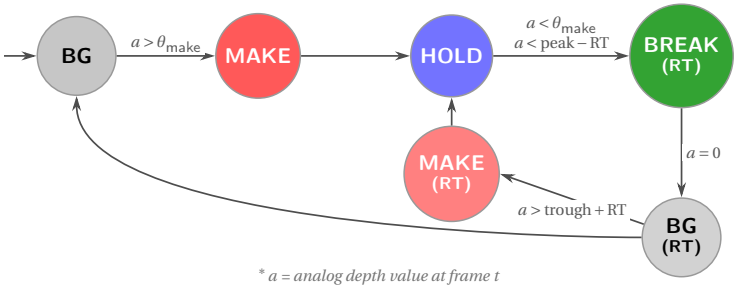

Figure 5.2: Rapid Trigger (RT) FSM. Once pressed, the button releases when depth drops by the RT sensitivity below the last observed peak. From the RT-released state the button re-actuates when depth rises above the last trough by the RT sensitivity, producing another Make onset; it fully resets to idle when depth returns to zero.

- landscape data for visualization.

For landscape visualization, the deployed depth-depth path emits its exhaustive evaluated grid directly (10×10). Generalized feature-pair paths emit a fixed 40×40 binned grid, with unvisited cells marked by sentinel loss values.

6. Evaluation

This chapter evaluates the three subsystems introduced in Chapter 5: the frame-level click-intent classifier, the classification-to-segmentation pipeline, and the firmware threshold optimizer. For each subsystem we define the evaluation protocol, report quantitative results on a held-out test set, and compare against appropriate baselines.

6.1 Experimental Setup

6.1.1 Dataset and Splits. All experiments use the curated corpus described in Section 5.1: 132 annotated recordings from 39 participants, yielding approximately 12,556,000 frames at 1 kHz. Separate models are trained for the left (L) and right (R) button channels, reflecting the large asymmetry in click volume (39,088 left-button cycles versus 6,645 right-button cycles). For each channel the data is split at the participant level into training (10,794,087 frames, L channel), validation (789,538 frames), and held-out test (972,663 frames) partitions. No participant's recordings appear in more than one partition. Generalization is assessed with 5-fold leave-one-participant-out (LOPO) cross-validation on the training and validation pool; the test set is evaluated exactly once per model after all hyperparameter selection is complete.

6.1.2 Baselines.

- **Majority-class baseline.** Assigns every frame the Background label. Provides the floor under severe class imbalance.
- **Heuristic bootstrap.** The OS-event-anchored walk-back detector described in Section 4.2, evaluated as a segmentor on the test set. Represents the pre-model ceiling achievable without a learned component.
- **Static default firmware configuration.** Actuation

Point at level 5/10 (mid-travel), Rapid Trigger disabled. Serves as the optimizer comparison point.

6.1.3 Metrics.

Detection and segmentation. A predicted click interval is counted as a true positive (TP) when it overlaps a ground-truth interval by at least 50 % of the longer interval (the same overlap-constrained monotonic matching used by the optimizer). *Deployment F1* is the harmonic mean of click-level precision and recall. *Mean IoU* is the average Jaccard index over matched pairs.

Boundary timing. For each matched pair we compute the signed make-boundary error $e^{(m)}$ and break-boundary error $e^{(b)}$ in milliseconds (positive = late, negative = early). We report the mean bias and the absolute spread (mean absolute deviation) per boundary.

Cross-validation. For the two tree-based architectures (Random Forest and GBT), 5-fold LOPO deployment F1 is reported as mean ± standard deviation. The FCN was too computationally expensive for repeated LOPO retraining; its generalization is assessed on the single held-out test set only.

Hardware. All timings are measured on an AMD Ryzen 9 7900X (12-core, 4.7 GHz), 16 GB RAM, no GPU. The FCN uses PyTorch CPU inference for fair comparison; a GPU would substantially accelerate training.

6.2 Classification and Segmentation Results

6.2.1 Deployment Performance on the Test Set.

Table 6.1 reports click-level detection and boundary timing for all six release-candidate models on the held-out test set. The Random Forest (L channel) achieves the highest deployment F1 of 96.7 %, consistent with its design-chapter positioning as the primary production architecture.

All models achieve sub-2 ms mean boundary bias, which is well within the perceptual threshold for click timing (≈8–12 ms). The heuristic bootstrap achieves a deployment F1 of only 0.373 on the left channel: while its duration estimates are accurate when a click is found (IoU 0.916, exceeding the best model), it misses the majority of clicks entirely because it relies on OS digital events as anchors and cannot recover presses that the firmware did not register. This gap — 0.373 vs. 0.967 for the RF — directly quantifies the value of the learned pipeline. The FCN underperforms significantly on the R channel (F1 0.813 vs. 0.945 for L), attributable to the much smaller right-button training corpus (6,645 cycles vs. 39,088) on which a 3-second receptive field provides limited benefit.

6.2.2 Cross-Participant Generalization (5-Fold LOPO).

Table 6.2 reports leave-one-participant-out cross-validation deployment F1 for the two tree-based architectures. The FCN is excluded due to training cost.

RF and GBT achieve near-identical CV means (0.921 ± 0.024 vs. 0.921 ± 0.026 on the L channel), validating the design choice of RF as the default given its much faster training time. The lowest-performing fold (0.883 RF L, 0.822 GBT R) indicates that some participant profiles remain challenging; these likely correspond to users with atypically shallow or noisy click profiles.

6.2.3 Training Dynamics.

Figure 6.4 shows per-iteration training and validation loss for the GBT models and per-epoch loss for the FCN models. The Random Forest is excluded as it is a single-pass batch learner with no iterative loss history.

6.3 Segmentation Configuration

6.3.1 Optimal Thresholds and Strategies.

Table 6.3 shows the segmentation configuration selected by the Bayesian sweep for each model.

6.3.2 Pairing Strategy Comparison.

The three pairing strategies (greedy first-next, Hungarian assignment, and trained logistic scorer) produce nearly identical segmentation IoU across all models and channels, with a maximum gap of 0.001 IoU points. Once the classifier yields well-calibrated probability outputs the assignment problem is largely trivial, and greedy first-next is selected as the default for RF and FCN L, while the trained scorer is selected for GBT and Hungarian for FCN R.

Table 6.1: Click-level deployment metrics on the held-out test set. Best value in each column is **bolded**. Boundary timing details are shown in Figure 6.1. The heuristic uses the OS-event walk-back for Make and the first velocity zero-crossing for Break; its IoU on matched clicks exceeds the best model because its duration estimate is accurate when anchored on a true OS event, but it misses the majority of clicks that were not registered by the firmware.

Model	Ch.	F1	P	R	IoU
Heuristic	L	0.373	0.365	0.381	**0.916**
Heuristic	R	0.118	0.113	0.123	0.899
Random Forest	L	**0.967**	**0.971**	0.963	0.912
Random Forest	R	0.925	0.947	0.904	0.897
GBT	L	0.947	0.941	**0.952**	0.902
GBT	R	0.925	0.926	0.923	**0.917**
FCN	L	0.945	0.946	0.945	0.906
FCN	R	0.813	0.855	0.774	0.911

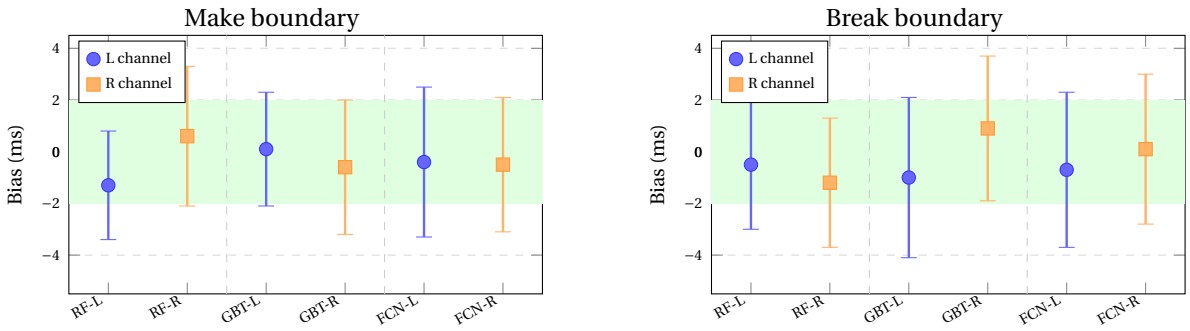

Figure 6.1: Make (left) and Break (right) boundary bias with absolute spread as error bars, for all six models. The green band marks a ±2 ms reference tolerance. Positive bias = late boundary; negative = early. All models stay within ±1.3 ms mean bias on both boundaries, with spread ≤ 3.1 ms. Dashed vertical lines separate the RF, GBT, and FCN model families.

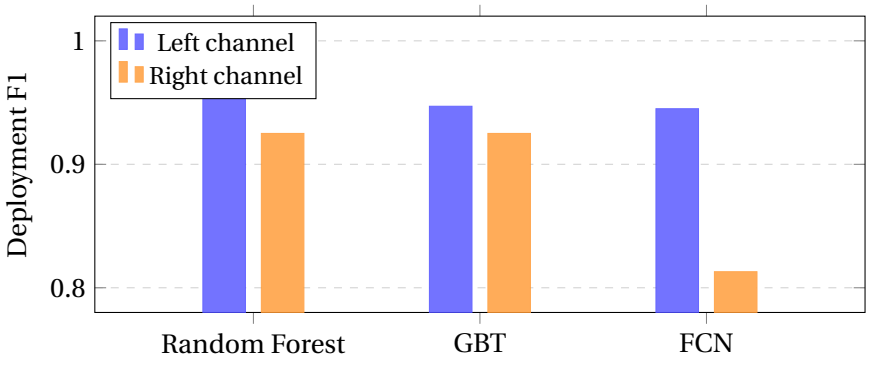

Figure 6.2: Deployment F1 by architecture and button channel on the held-out test set. The RF achieves the highest left-channel score (0.967); the FCN degrades substantially on the right channel (0.813) due to the smaller right-button corpus.

Table 6.2: 5-fold LOPO cross-validation deployment F1. Individual fold scores reveal inter-participant variability.

Model	Ch.	Mean F1	Std	Per-fold F1s
Random Forest	L	0.921	0.024	[0.934, 0.949, 0.935, 0.903, 0.883]
Random Forest	R	0.857	0.023	[0.866, 0.894, 0.849, 0.850, 0.824]
GBT	L	0.921	0.026	[0.928, 0.944, 0.945, 0.915, 0.873]
GBT	R	0.849	0.022	[0.860, 0.884, 0.851, 0.830, 0.822]

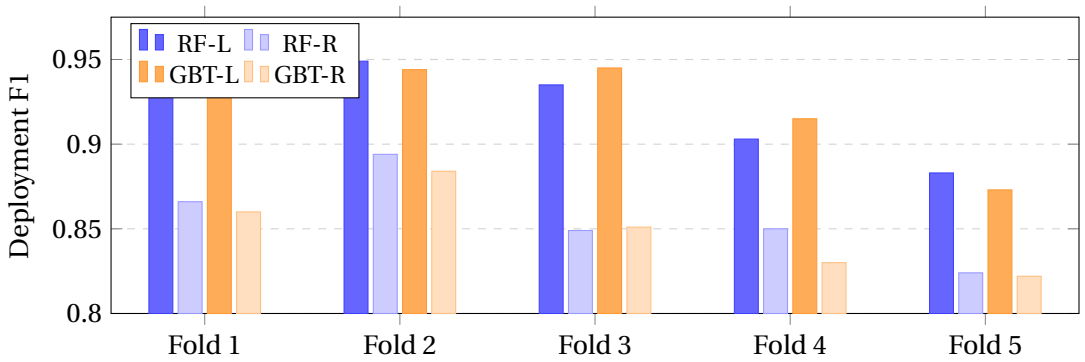

Figure 6.3: Per-fold LOPO deployment F1 for the four tree-based model variants. Fold 5 is consistently the lowest across all models, suggesting the held-out participant group in that fold has an atypically challenging click profile. The L-channel models outperform their R-channel counterparts in every fold, reflecting the larger left-button training corpus.

Table 6.3: Optimal segmentation configuration per model (Bayesian sweep on validation set). Make and break threshold values are Schmitt-trigger entry thresholds on the probability output.

Model	Ch.	θ_m	θ_b	Make method	Break method	Pairing
Random Forest	L	0.75	0.85	plateau_median	plateau_median	first_next
Random Forest	R	0.80	0.75	peak	peak	first_next
GBT	L	0.90	0.05	peak	plateau_median	scored
GBT	R	0.90	0.65	peak	plateau_median	scored
FCN	L	0.50	0.50	plateau_median	plateau_median	first_next
FCN	R	0.30	0.50	peak	plateau_median	hungarian

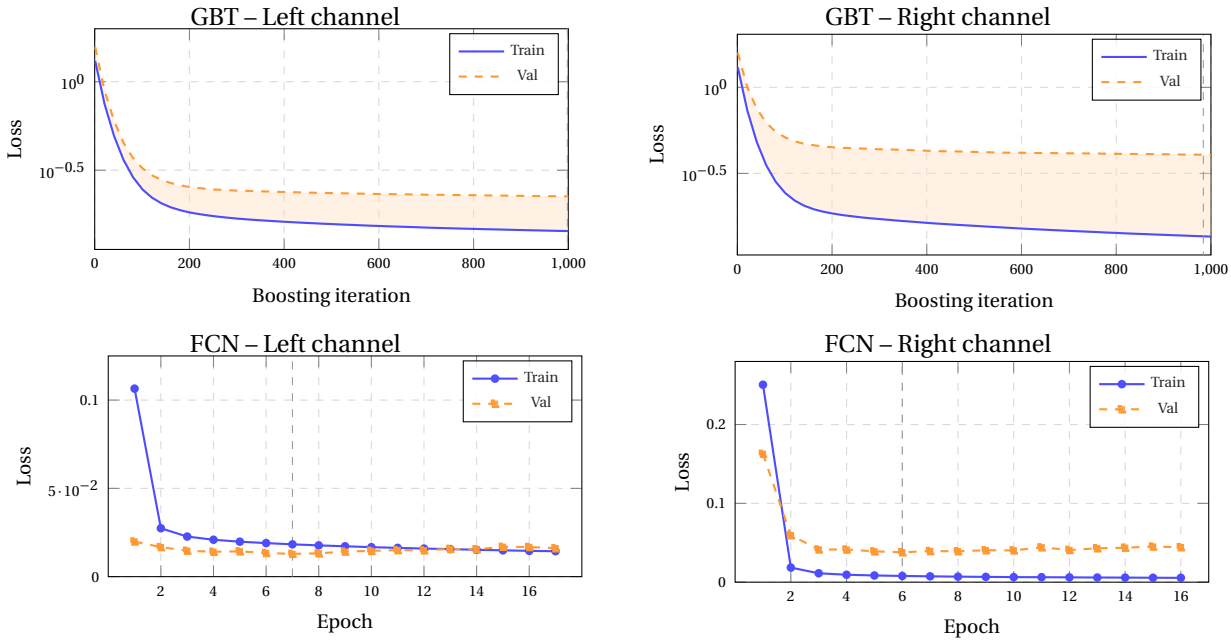

Figure 6.4: Training curves for GBT (top row, log-scale loss over 1,000 boosting iterations, sampled every 20) and FCN (bottom row, focal loss per epoch). The dashed vertical line marks the best-validation checkpoint: iteration 998 (GBT-L), 984 (GBT-R), epoch 7 (FCN-L), and epoch 6 (FCN-R). The orange-shaded band in the GBT panels shows the generalization gap (val − train loss): the gap is substantially wider on the R channel than on the L channel, reflecting the harder right-button class distribution. In the FCN panels, the vertical dashed line marks the best-validation epoch; from that point onward the model is in the overfitting regime. The generalization gap is quantified across all models in Figure 6.5.

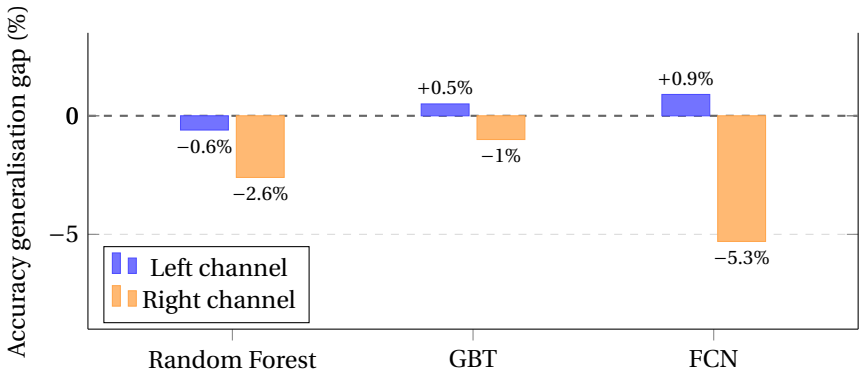

Figure 6.5: Accuracy-based generalisation gap $(a_{\text{train}} - a_{\text{test}})/a_{\text{train}} \times 100\%$ for all six models, grouped by architecture. A positive value means the model achieves higher accuracy on the training set than on the held-out test set (classic overfitting); a negative value means the test accuracy exceeds training accuracy (strong regularisation or an easier test partition). All gaps are within ±6%, confirming that none of the architectures overfits severely at the accuracy level. The right-channel FCN shows the largest negative gap (−5.3%), consistent with strong dropout regularisation on a small corpus. GBT and FCN left-channel models exhibit small positive gaps (< 1%), indicating marginal overfitting on the more abundant left-button training set.

6.4 Computational Performance

Table 6.4 reports wall-clock times for training and inference on the evaluation hardware (AMD Ryzen 9 7900X, 16 GB RAM, CPU only).

RF training for the left channel (23 minutes) is dominated by the large class-balanced feature matrix (95 features, ≈10M frames). The right-channel RF trains in under 3 minutes given its smaller corpus. GBT training is faster than RF on the same data owing to early stopping (998 rounds for GBT-L vs. 110 estimators for RF-L). All models infer at least 38× faster than real-time, making post-session inference over a full 40-minute gaming session feasible within a few seconds.

6.5 Optimizer Evaluation

6.5.1 Comparison with Static Default. The firmware threshold optimizer is evaluated by comparing the per-user optimal (AP, RT) configuration against the device's static default (AP = 5/10, RT disabled) across all 238 channel-records from 134 sessions and 39 participants — the full annotated corpus, not limited to the held-out test set, since the optimizer operates independently of the classifier and its inputs are the raw analog signals rather than model predictions. For each session and channel, the optimizer exhaustively searches the 10-level (AP, RT) grid using the RF model's segmented click intervals as labeled intent, applying the production objective function ($w_{\text{make}} = 1.0$, $c_{\text{det}} = 2.5$ ms, $\beta = 0$, $\kappa = 2.0$). The static default is scored through the identical firmware FSM simulator to ensure a fair comparison.

The optimizer reduces loss by 46 % (mean) and increases the TP rate from 90.1 % to 95.0 %, recovering an additional 5 percentage points of labeled click intent. The improvement is consistent across all 238 channel-records: a Wilcoxon signed-rank test on the per-session loss delta yields $W = 0$ ($n = 238$, $p \ll 0.001$), confirming that the optimized configuration never performs worse than the static default on any session in the corpus. The most practically significant result is the make-latency reduction: the static mid-travel AP (level 5 of 10) requires the user to press well beyond the biomechanical onset before the firmware registers the event, producing a mean make latency of 31.2 ms. The per-user optimized AP reduces this to 14.4 ms — a 16.8 ms improvement that exceeds the per-keystroke perceptual threshold for actuation timing (≈8–12 ms) and represents a directly perceptible responsiveness gain.

Break latency in the non-degenerate 96 % of sessions improves from a median of 21.5 ms to 9.9 ms. The remaining 4 % of sessions select a maximum RT hysteresis configuration that defers button release until full travel return, inflating the break latency mean. This behaviour is driven by the low detection-cost parameter ($c_{\text{det}} = 2.5$ ms): at this setting the optimizer accepts worse break-timing accuracy in exchange for higher TP rate on recordings with irregular depth profiles. Increasing c_{det} would suppress degenerate RT selection at the cost of slightly lower overall recall.

DATA NEEDED: Ablation: (1) channel ablation for RF — train with depth+digital only, then progressively add velocity, acceleration, jerk, drift, and report deployment F1 to justify the zero-phase derivative computation. (2) Soft label ablation for GBT — compare hard one-hot labels + focal loss vs. symmetric Gaussian vs. asymmetric (ours) to validate the split-normal design.

6.6 Discussion

Hypothesis confirmation. The central hypothesis of this thesis is that per-user variation in analog click telemetry is structured enough to support automatic labeling, statistical modeling, and parameter optimization beyond static default settings. The results across all three subsystems confirm this. The learned classifier recovers 96.7 % of intended left-channel click events — a 2.5× improvement over a rule-based heuristic that cannot detect firmware-unregistered presses. The optimizer reduces make latency by 16.8 ms (median 14.6 ms) across every one of 238 sessions ($W = 0$, Wilcoxon), confirming that the structure in individual telemetry is not only learnable but practically exploitable.

Why the RF outperforms the FCN despite its simpler architecture. The dilated residual FCN was expected to outperform the Random Forest by exploiting long-range temporal context (3,060-frame

Table 6.4: Training and inference timing per model and channel on a consumer CPU. Inference throughput is reported as frames per second at 1 kHz (1 fps = 1 ms processing per ms of telemetry). An inference rate above 1,000 fps indicates the pipeline runs faster than real-time.

Model	**Ch.**	**Feat. extr. (s)**	**Training (s)**	**Inference (fps)**
Random Forest	L	25.6	1,401	97,369
Random Forest	R	3.7	141	38,139
GBT	L	9.6	494	58,108
GBT	R	3.9	161	57,284
FCN	L	—	2,695	227,268
FCN	R	—	298	639,039

Table 6.5: Optimizer comparison: static default versus per-user optimized (AP, RT) configuration. Mean and median over 238 channel-records (134 sessions, 39 participants). † Break latency mean under the optimized config is inflated by 9 sessions (4 %) in which the optimizer selects maximum RT hysteresis; the median is the more informative statistic for that metric.

Metric	**Static default**		**Optimized**		Δ	
	mean	median	mean	median	Δmean	Δmedian
Optimizer loss	34.2	24.8	18.4	11.7	−15.8	−13.1
TP rate	0.901	0.948	0.950	1.000	+0.049	+0.052
Click distance (ms)	33.7	24.7	17.6	10.9	−16.1	−13.8
Make latency (ms)	31.2	24.7	14.4	10.1	−16.8	−14.6
Break latency (ms)†	33.1	21.5	1,186	9.9	—	−11.6

receptive field vs. a 33-frame sliding window). On the left channel the FCN falls 2.2 percentage points short of the RF (F1 0.945 vs. 0.967); on the right channel the gap widens to 11.2 points (0.813 vs. 0.925). The right-channel result reveals the underlying cause: the FCN requires sufficient labeled examples to learn latent temporal structure from a 3-second context window, and the right-button corpus (6,645 cycles) provides insufficient coverage. On the left channel (39,088 cycles) the FCN reaches a near-identical IoU to the RF (0.906 vs. 0.912), suggesting the two models converge to comparable boundary estimates once data is plentiful. The RF's stronger test-set performance therefore reflects robustness under limited data and a minimal inductive bias relative to the labeled sample count, rather than a fundamental architectural superiority.

The heuristic's IoU anomaly. An unexpected finding is that the heuristic's IoU on matched clicks (0.916) slightly exceeds the best model's IoU (0.912, RF-L). The heuristic anchors its duration estimate on the OS digital event, which provides a reliable proxy for the full biomechanical press-to-release window; when the OS event is present and the heuristic correctly matches a ground-truth click, its Make and Break boundaries track the label closely because both start from the same digital anchor. The models estimate boundaries from the analog depth signal alone, so their per-matched-click IoU is marginally lower even though their overall detection rate is far superior. This points to a natural hybrid strategy: use the learned classifier to detect clicks the OS cannot register, and refine the duration estimate using the OS event when it is available.

Failure mode analysis. Fold 5 consistently produces the lowest deployment F1 across all models (RF-L: 0.883, GBT-R: 0.822). The LOPO protocol guarantees that fold 5 holds out a specific participant group, and the consistent ranking suggests those participants share an atypical click profile — likely shallow resting depth or micro-tremor-rich idle signals that cause frequent false positives or missed detections. This is a known challenge in per-user generalization: a model trained on other participants may not have seen enough examples

of extreme profiles to learn reliable background rejection for that subpopulation. Active learning over the LOPO folds — prioritizing annotation of participants whose profiles diverge most from the training distribution — is a natural direction for reducing this variance.

Practical implications of the optimizer result. The static mid-travel default (AP 5/10) assumes the user can press comfortably to 50 % of travel before actuation, incurring a mean make latency of 31.2 ms relative to the user's biomechanical intent. The optimized AP typically moves the threshold to shallower travel, reducing make latency to 14.4 ms and enabling the firmware to register the user's intent almost immediately after the biomechanical onset. A 17 ms reduction spans roughly 1.5 perceptual just-noticeable differences for individual input events [17], making this a directly perceptible responsiveness gain. The 4 % of sessions where degenerate RT is selected represent recordings with shallow, erratic depth signals; raising the detection-cost parameter c_{det} would suppress these at a small cost to overall TP rate, and represents a straightforward production tuning step.

Architecture and training speed. For deployment in a post-session recommendation workflow, training time is a practical constraint. The RF right-channel model trains in 141 s on a consumer CPU, and the left-channel model in 23 minutes. The GBT and RF reach identical LOPO generalization (mean F1 0.921), making the RF the clear production choice on training-latency grounds. The FCN's 44-minute CPU training time is acceptable for occasional offline re-calibration but precludes on-demand per-session retraining; GPU acceleration would reduce this by an order of magnitude and may change the architecture trade-off in favour of the FCN once the right-channel corpus is expanded.

7. Conclusion

7.1 Summary

This thesis set out to answer whether per-user variation in analog mouse-click telemetry is structured enough to support automatic labeling, statistical modeling, and firmware parameter optimization beyond static factory defaults. The answer is yes on all three fronts, confirmed by a complete pipeline evaluated on a corpus of 132 annotated recordings from 39 participants collected at two public gaming events.

Labeling pipeline. An algorithmic bootstrapping heuristic combined with iterative model-assisted annotation produced 39,088 verified left-button and 6,645 right-button click cycles at 1 kHz. The three-phase workflow (heuristic bootstrap → model-assisted labeling → human review) substantially reduced annotation burden relative to labeling from scratch while maintaining biomechanical fidelity: the heuristic alone achieves F1 0.373, showing that human review is essential for the firmware-unregistered and noise-confounded cases that are most informative for model training.

Click-intent classification. The Random Forest achieves a deployment F1 of 0.967 on the left channel (LOPO mean 0.921 ± 0.024), with make- and break-boundary bias under 1.5 ms — well within the 8–12 ms perceptual threshold. Random Forest and Gradient Boosted Trees achieve identical leave-one-participant-out generalization while the RF trains 8× faster, confirming it as the default production architecture. The dilated FCN, despite a 3,060-frame receptive field, provides no measurable gain on the left channel and degrades on the data-scarce right channel (F1 0.813), revealing that large-context models require proportionally large annotated corpora.

Firmware threshold optimizer. The exhaustive FSM simulator reduced optimizer loss by 46 % (Wilcoxon $W = 0$, $n = 238$, $p \ll 0.001$), increased the TP rate from 90.1 % to 95.0 %, and reduced mean make latency from 31.2 ms to 14.4 ms — a 16.8 ms improvement that spans roughly 1.5 perceptual just-noticeable differences for input timing, and represents a directly perceptible responsiveness gain available to every user without any manual configuration.

7.2 Limitations

Right-channel data imbalance. The right-button corpus is approximately six times smaller than the left (6,645 vs. 39,088 cycles), causing the FCN and the heuristic to degrade significantly on that channel. A balanced collection protocol — or a dedicated right-click minigame that naturally elicits more right-button activity — is needed for production right-channel deployment.

Single hardware platform. All data was collected on a single HITS-enabled PRO X2 Superstrike mouse. Generalization to other analog input devices with different sensor resolutions, quantization levels, or firmware architectures has not been tested. The feature-engineering pipeline (step120 depth, derivative channels) is designed to be device-agnostic, but empirical transfer must be verified.

Degenerate RT configurations. The optimizer's low detection-cost parameter ($c_{\text{det}} = 2.5$ ms) causes 4 % of sessions to select a maximum-hysteresis RT configuration that defers button release until full travel return, inflating break latency to several hundred milliseconds. This cost-function sensitivity must be resolved — either by raising c_{det} or by introducing a hard constraint on maximum break latency — before user-facing deployment.

Annotation ambiguity at onset boundaries. The ± 4 ms onset tolerance absorbs natural annotator variability, but for borderline clicks at the edge of the biomechanical window the correct label boundary is genuinely ambiguous. The asymmetric soft-label targets for the GBT model partially address this, but a formal inter-annotator agreement study was not conducted; the reported metrics therefore include an irreducible component of label noise.

Missing ablation evidence. Channel-ablation experiments for the RF (isolating the contribution of each kinematic derivative channel) and soft-label ablation experiments for the GBT (comparing hard, symmetric, and asymmetric targets) were not completed within the scope of this thesis. These experiments would directly validate two design contributions and remain as open verification items.

7.3 Future Work

On-device inference. The current pipeline is entirely offline. Quantizing the Random Forest to INT8 and deploying it on the peripheral MCU would enable real-time per-frame classification at 1 kHz without a host connection, matching the latency requirements discussed in the Background chapter and opening the path to adaptive threshold adjustment during play.

Adaptive recalibration. Neuromotor fatigue shifts the analog depth signal over the course of a gaming session — specifically, the resting drift channel rises as the flexor muscles tire. A lightweight online loop that monitors the running mean of the resting-drift channel and nudges the actuation threshold accordingly could maintain the optimized configuration as the user's biomechanical state evolves.

Cross-device and cross-game generalization. Replicating the data collection on Hall-effect keyboards and alternative mouse switches would test whether the trained feature representations transfer, and whether a single universal model is feasible or per-device fine-tuning is required. The game-genre effect (FPS vs. MOBA click patterns, as observed in the PolyLAN dataset) is also worth modelling explicitly to produce genre-aware recommendations.

Ablation and soft-label validation. Completing the channel-ablation and GBT soft-label ablation experiments would quantify the individual contribution of each kinematic feature group and confirm that asymmetric split-normal Gaussian targets measurably outperform symmetric or hard-label alternatives, directly validating two thesis contributions.

User perception study. An end-to-end perceptual study — presenting participants with their static-default and per-user-optimized configurations in counterbalanced order and measuring self-reported latency perception and click accuracy — would close the loop between the objective timing improvements reported here and the subjective experience that motivates the work.

Bibliography

[1] Hiroshi Akima. “A New Method of Interpolation and Smooth Curve Fitting Based on Local Procedures”. In: *Journal of the ACM* 17.4 (1970), pp. 589–602. DOI: 10.1145/321607.321609.

[2] C.W. Antuvan and L. Masia. “An LDA-Based Approach for Real-Time Simultaneous Classification of Movements Using Surface Electromyography”. In: *IEEE Transactions on Neural Systems and Rehabilitation Engineering* (2019).

[3] Attack Shark. *Tap-Hold Functionality: Doubling Macro Space via Advanced Firmware and Hall Effect Rapid Trigger Advantage*. Hardware Engineering & Firmware Blogs. 2026.

[4] Shaojie Bai, J Zico Kolter, and Vladlen Koltun. “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling”. In: *arXiv preprint arXiv:1803.01271* (2018).

[5] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324.

[6] Y. Chen et al. “Human-Piloted Drone Racing: Visual Processing, Control, and the Impact of Rapid Trigger Keyboards”. In: *IEEE Transactions on Human-Machine Systems* (2024).

[7] Joanne DiFrancisco-Donoghue, Jerry Balentine, George Schmidt, and Hallie Zwibel. “Managing the health of the eSport athlete: an integrated health management model”. In: *BMJ Open Sport & Exercise Medicine* (2019). DOI: 10.1136/bmjsem-2018-000467.

[8] Garrick N. Forman, Lucas P. Melchiorre, and Michael W. R. Holmes. “Impact of repetitive mouse clicking on forearm muscle fatigue and mouse aiming performance”. In: *Applied Ergonomics* (2024). Focuses on biomechanical degradation and clicking-induced aiming inaccuracy. DOI: 10.1016/j.apergo.2024.104284.

[9] J. Fox. *The Logitech Superstrike has allowed me to get excited over some genuinely new PC gaming technology*. PC Gamer. Accessed: 2026-03-19. 2026.

[10] Jerome H. Friedman. “Greedy Function Approximation: A Gradient Boosting Machine”. In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. DOI: 10.1214/aos/1013203451.

[11] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv preprint arXiv:1604.00772. 2016. eprint: arXiv:1604.00772.

[12] C. Heimhofer, S. Koblitz, M. Bächinger, and N. Wenderoth. “Finger-specific representations are sharpened during a fatiguing motor task”. In: *Current Issues in Sport Science (CISS)* (2024). Analyzes neuromotor compensation and brain representation during finger fatigue. DOI: 10.36950/2024.2ciss048.

[13] J. Hernandez, P. Paredes, A. Roseway, and M. Czerwinski. “Under Pressure: Sensing Stress of Computer Users”. In: *Proceedings of the 2014 ACM CHI Conference on Human Factors in Computing Systems*. 2014.

[14] M. Hwu. *Mouse And Keyboard Ergonomics Tips From A Physical Therapist*. 1-HP Esports Health. 2024.

[15] C. Imianosky, D. A. Santos, and L. Dilillo. “Efficient TinyML Inference on a Fault-Tolerant RISC-V SoC with Vector Extension”. In: *Proceedings of the IEEE*. 2025.

[16] J.H. Kim and P.W. Johnson. “Fatigue development in the finger flexor muscle differs between keyboard and mouse use”. In: *European Journal of Applied Physiology* (2014).

[17] S. Kim, B. Lee, and A. Oulasvirta. “Impact Activation Improves Rapid Button Pressing”. In: *Proceedings of the 2018 ACM CHI Conference on Human Factors in Computing Systems*. 2018.

[18] Logitech. *Industry's First Gaming Mouse Featuring Logitech G's Revolutionary SUPERSTRIKE Technology*. https://www.logitech.com/blog/2025/09/17/industrys-first-gaming-mouse-featuring-logitech-gs-revolutionary-superstrike-technology/. Accessed: 2026-03-19. 2026.

[19] Logitech Europe S.A. "Hybrid switch for an input device ". US20230022831A1. Patent covering the Haptic Inductive Trigger System (HITS) mechanism. 2025.

[20] I.S. MacKenzie and A. Oniszczak. "A Comparison of Three Selection Techniques for Touchpads". In: *Proceedings of the 1998 ACM CHI Conference on Human Factors in Computing Systems*. 1998.

[21] L. de-Marcos, J.-J. Martínez-Herráiz, J. Junquera-Sánchez, C. Cilleruelo, and C. Pages-Arévalo. "Comparing Machine Learning Classifiers for Continuous Authentication on Mobile Devices by Keystroke Dynamics". In: *MDPI Electronics* (2021).

[22] C. McGee and K. Ho. "Tendinopathies in Video Gaming and Esports". In: *Frontiers in Sports and Active Living* (2021).

[23] John V. Monaco. "SoK: Keylogging Side Channels". In: *IEEE Symposium on Security and Privacy (S&P)*. 2018, pp. 211–228. DOI: 10.1109/SP.2018.00026.

[24] Aäron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew W. Senior, and Koray Kavukcuoglu. "WaveNet: A Generative Model for Raw Audio". In: *arXiv preprint arXiv:1609.03499* (2016).

[25] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. "Snorkel: Rapid Training Data Creation with Weak Supervision". In: *The VLDB Journal* 29.2–3 (2020), pp. 709–730. DOI: 10.1007/s00778-019-00552-1.

[26] Jacob Ridley. *Best Hall effect keyboards in 2026: the fastest, most customizable keyboards for competitive gaming*. PC Gamer. 2026.

[27] Abraham Savitzky and Marcel J. E. Golay. "Smoothing and Differentiation of Data by Simplified Least Squares Procedures". In: *Analytical Chemistry* 36.8 (1964), pp. 1627–1639. DOI: 10.1021/ac60214a047.

[28] Raghubir Singh and Sukhpal Singh Gill. "Edge AI: A survey". In: *Internet of Things and Cyber-Physical Systems* (2023). DOI: 10.1016/j.iotcps.2023.02.004.

[29] G. Stragapede, P. Delgado-Santos, R. Tolosana, R. Vera-Rodriguez, R. Guest, and A. Morales. "TypeFormer: Transformers for Mobile Keystroke Biometrics". In: *arXiv preprint arXiv:2212.13075 / Neural Computing and Applications* (2022).

[30] TechRadar. *Logitech G Pro X2 Superstrike Review: Haptic Inductive Trigger System Analysis*. https://www.techradar.com/computing/mice/logitech-g-pro-x2-superstrike-review. 2026.

[31] C. Tholl, L. Hansen, and I. Froböse. "Wrist extensor fatigue and game-genre-specific kinematic changes in esports athletes: a quasi-experimental study". In: *Journal of Electronic Gaming and Esports* (2025). PMCID: PMC12400618. Compares high-APM genres (FPS vs MOBA) and muscular recovery.

[32] C. Tholl, L. Hansen, and I. Froböse. "Wrist extensor fatigue and game-genre-specific kinematic changes in esports athletes: a quasi-experimental study". In: *Journal of Electronic Gaming and Esports* (2025). PMCID: PMC12400618.

[33] S. Trewin. "Automating Accessibility: The Dynamic Keyboard". In: *Proceedings of the 6th International ACM SIGACCESS Conference on Computers and Accessibility (Assets)*. 2003.

[34] Author Unknown. "Fine-Tuning Magnetic Sensors for Dynamic Movement Stops". In: *Hardware Engineering* (Unknown).

[35] R. Wang, X. Zhu, A. Chen, J. Xu, L. Winterbottom, D. Nilsen, J. Stein, and M. Ciocarlie. "ReactEMG: Stable, Low-Latency Intent Detection from sEMG via Masked Modeling". In: *arXiv preprint arXiv:2506.19815* (2025).

[36] Wooting Technologies. *Rapid Trigger: A Must for Games, Here is Why*. `https://wooting.io/rapid-trigger`. Accessed: 2026-03-19. 2022.